

Evidence of successful completion of the activities and learning.

Exercise 3.2.1

Why do we need transport layer protocols?

We do need them as they allow process to process communication between applications running on different hosts. They handle things like multiplexing/demultiplexing using port numbers, and depending on the protocol, they can also provide reliability, flow control and congestion control (in TCP).

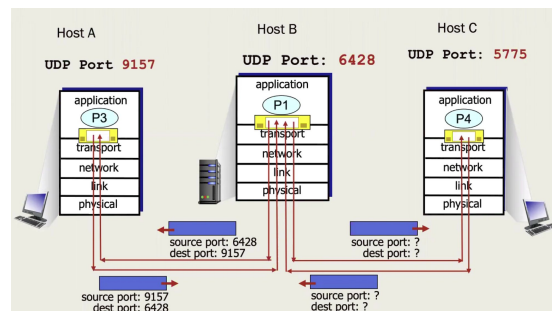
Can the Internet survive without transport layer protocols?

IP could still route packets between different hosts, but there would be no standard mechanism set to deliver the received data to the correct application process. Also, there would be no TCP reliability, ordering, flow/congestion control features. So, the network layer could still technically operate, but most internet applications will not function properly, unless another alternative standard is set as a replacement and both the client and server agrees upon it.

Exercise 3.2.2

Determine the source and destination ports of the transport layer segments that are sending from Hosts B to C and C to B, in the connectionless multiplexing and demultiplexing example that we discussed in the above video.

- B to C:
 - o Source Port: 6428
 - o Destination Port: 5775
- C to B:
 - o Source Port: 5775
 - o Destination Port: 6428



Exercise 3.4.1

I first filtered out this TCP connection.

The image shows a Wireshark packet capture window with a filter set to 'tcp'. The packet list shows several packets, with packet 13 selected. The packet details pane shows the structure of the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header. The packet bytes pane shows the raw data of the packet.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
13	1.640530809	0.000062868	192.168.8.136	34.36.133.15	TCP	74	50576 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1523368
14	1.648442343	0.007911534	23.206.199.208	192.168.8.136	TLSv1...	90	Application Data
15	1.648464023	0.000021680	192.168.8.136	23.206.199.208	TCP	54	45242 → 443 [RST] Seq=2 Win=0 Len=0
16	1.648595750	0.000131727	23.206.199.208	192.168.8.136	TCP	66	443 → 45242 [FIN. ACK] Seq=25 Ack=2 Win=567 Len=0 TSval=3916014852 TSecr

Frame 13: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface enp9s0, id 0

Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)

Internet Protocol Version 4, Src: 192.168.8.136, Dst: 34.36.133.15

Transmission Control Protocol, Src Port: 50576, Dst Port: 443, Seq: 0, Len: 0

Source Port: 50576

Destination Port: 443

Exercise 4.3.2

I will make this a continuation of Exercise 4.3.1.

From here, I got the port number, which was 50576. To get rid of all the other noise, I then filtered the TCP traffic using `tcp.port == 50576`. After that, I was able to clearly identify the three-way handshake that had occurred.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
13	1.640530809		192.168.8.136	34.36.133.15	TCP	74	50576 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1523368856 TSecr=0 WS=1024
20	1.651787519	0.011256710	34.36.133.15	192.168.8.136	TCP	74	443 → 50576 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=431428324 TSecr=1523368856 WS=256
21	1.651798740	0.000011221	192.168.8.136	34.36.133.15	TCP	66	50576 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1523368886 TSecr=431428324

No.	Time	Delta	Source	Destination	Protocol	Length	Info
20	1.651787519	0.011256710	34.36.133.15	192.168.8.136	TCP	74	443 → 50576 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=431428324 TSecr=1523368856 WS=256
21	1.651798740	0.000911221	192.168.8.136	34.36.133.15	TCP	66	50576 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1523368868 TSecr=431428324
22	1.652034632	0.000235892	192.168.8.136	34.36.133.15	TLShello	2116	Client Hello (SNI=ollama.com)

```

Frame 20: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface enp9s0, id 0
  Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:b2:e2:0b:8d:70)
  Internet Protocol Version 4, Src: 34.36.133.15, Dst: 192.168.8.136
  Transmission Control Protocol, Src Port: 443, Dst Port: 50576, Seq: 0, Ack: 1, Len: 0
    Source Port: 443
    Destination Port: 50576
    [Stream index: 4]
    [Stream Packet Number: 2]
    [Conversation completeness: Incomplete, DATA (15)]
    [TCP Segment Len: 0]
    Sequence Number: 0 (relative sequence number)
    Sequence Number (raw): 2535765808
    [Next Sequence Number: 1 (relative sequence number)]
    Acknowledgment Number: 1 (relative ack number)
    Acknowledgment number (raw): 3499858638
    1010 .... = Header Length: 40 bytes (10)
    ... 0000 0001 0010 = Flags: 0x012 (SYN, ACK)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0x0bb0 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
    [Timestamps]
    [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]

```

No.	Time	Delta	Source	Destination	Protocol	Length	Info
20	1.651787519	0.011256710	34.36.133.15	192.168.8.136	TCP	74	443 → 50576 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=431428324 TSecr=1523368856 WS=256
21	1.651798748	0.000011221	192.168.8.136	34.36.133.15	TCP	66	50576 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1523368868 TSecr=431428324
22	1.652034632	0.000235892	192.168.8.136	34.36.133.15	TLShello	2116	Client Hello (SNI=ollama.com)

```
> Frame 21: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 34.36.133.15
✓ Transmission Control Protocol, Src Port: 50576, Dst Port: 443, Seq: 1, Ack: 1, Len: 0
  Source Port: 50576
  Destination Port: 443
  [Stream index: 4]
  [Stream Packet Number: 3]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3499858638
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 2535765809
  1000 .... = Header Length: 32 bytes (8)
  > ... 0000 0001 0000 = Flags: 0x010 (ACK)
  Window: 63
  [Calculated window size: 64512]
  [Window size scaling factor: 1024]
  Checksum: 0x708a [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
```

No.	Time	Delta	Source	Destination	Protocol	Length	Info
20	1.651787519	0.011256710	34.36.133.15	192.168.8.136	TCP	74	443 → 50576 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=431428324 TSecr=1523368856 WS=256
21	1.651798740	0.00011221	192.168.8.136	34.36.133.15	TCP	66	50576 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1523368868 TSecr=431428324
22	1.652034632	0.000235892	192.168.8.136	34.36.133.15	TLShv1	2116	Client Hello (SNI=ollama.com)

Frame 22: Packet, 2116 bytes on wire (16928 bits), 2116 bytes captured (16928 bits) on interface enp9s0, id 0
 Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
 Internet Protocol Version 4, Src: 192.168.8.136, Dst: 34.36.133.15
 Transmission Control Protocol, Src Port: 50576, Dst Port: 443, Seq: 1, Ack: 1, Len: 2050

Source Port: 50576
 Destination Port: 443
 [Stream index: 4]
 [Stream Packet Number: 4]
 [Conversation completeness: Incomplete, DATA (15)]
 [TCP Segment Len: 2050]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 3499858638
 [Next Sequence Number: 2051 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 2535765809
 1000 = Header Length: 32 bytes (8)
 0000 0001 1000 = Flags: 0x018 (PSH, ACK)
 Window: 63
 [Calculated window size: 64512]
 [Window size scaling factor: 1024]
 Checksum: 0x788c [unverified]
 [Checksum Status: Unverified]
 Urgent Pointer: 0
 Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
 [Timestamps]
 [SEQ/ACK analysis]
 [Client Contiguous Streams: 1]
 [Server Contiguous Streams: 1]
 TCP payload (2050 bytes)

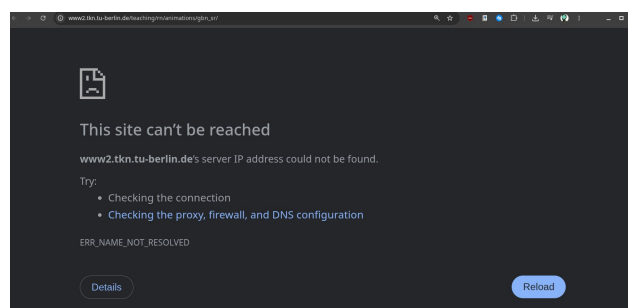
Transport Layer Security

I captured the TCP three way handshake between my host using port 50576 and the server on port 443. The first segment was a SYNC from 50576 to 443 with the raw sequence number 3499858637 followed by a SYN/ACK from 443 to 50576 with raw sequence number 2535765808 and acknowledgment number 3499858638. The final segment was an ACK from 50576 to 443 with the sequence number 3499858638 and acknowledgment number 2535765809, thus completing the connection setup. The TCP headers also contained connections like the header length, window size, checksum, urgent pointer and other TCP options. The SYN and SYN/ACK segments had a header of 40 bytes size, whereas the final ACK used a 32 byte header. You can download this file here. [Click Here](#).

The raw sequence numbers here also clearly show the flow of messages between the client and the server since each acknowledgment points to the next sequence number expected from the other side. The increment by 1 during the handshake also shows that the SYNC itself takes up one sequence number.

To answer the asked question, yes, I was able to identify the three way handshake. Packet 13 was the initial SYN from 192.168.8.136:50576 to 34.36.133.15:443, packet 20 was the SYN/ACK response from the server, and packet 21 was the final ACK from the client. The flags were set correctly as SYN=1, then SYN=1 and ACK=1, followed by ACK=1, thus confirming the TCP connection was established successfully.

Exercise 3.5.1



The given URL refused to work. Therefore, I opened the last snapshot of this website that I could find on The Way Back Machine: https://web.archive.org/web/20260309065145/https://www2.tkn.tu-berlin.de/teaching/rn/animations/gbn_sr/

Selective Repeat / Go Back N

configuration

protocol

☒ Go back N
☐ Selective Repeat

choosing a new protocol restarts the simulation

window size

8

sets the window size for the windows

end to end delay

12000

time a packet takes from one station to the other

timeout

16500

scroll mode

Typewriter style

change the style the window scrolls

number of packets emitted per minute

60

the number of packets the upper layer tries to send per minute

automatic emission of packets

start

starts or stops the automatic emission of packets by the upper layer

legend

- no data received yet
- data buffered (ready to send, delivered or sent but no ack received yet)
- ack
- transmission confirmed
- data has been delivered to upper network layer

coded by Johannes Kessler 2012

The main difference here is how they handle lost packets^{[1][2]}.

- With Go-Back-N, if a packet is lost, the sender retransmits that packet and the following unacknowledged packets in the window. The receiver can discard the out of order packets, which keeps the implementation much much simpler.
- With Selective Repeat, correctly received out of order packets are buffered and only the missing packet needs to be retransmitted.

This major difference makes Selective Repeat more efficient, but it also needs more buffering and tracking at the receiver, since it only retransmits the specific packets that were lost or received incorrectly. Go-Back-N may retransmit multiple packets after a loss, including packets that were already received successfully – wasting more bandwidth. This difference is said to become very noticeable when the window size increases^[2].

Week 4 – Activity 1

1. What method would you take if your friend needs to pack everything within an hour because the moving truck will be there soon?

I would pack everything ASAP without spending too much time organizing or checking every single item. I would prioritize my speed here.

2. How would your method change if you have more time and your friend would like to make sure each item in the kitchen is safely delivered, and your friend wants to keep those on the same order it was on the previous house?

If I had more time, I would pack everything carefully, label and organize the items, and ensure they are delivered safely and is kept in the same order as before.

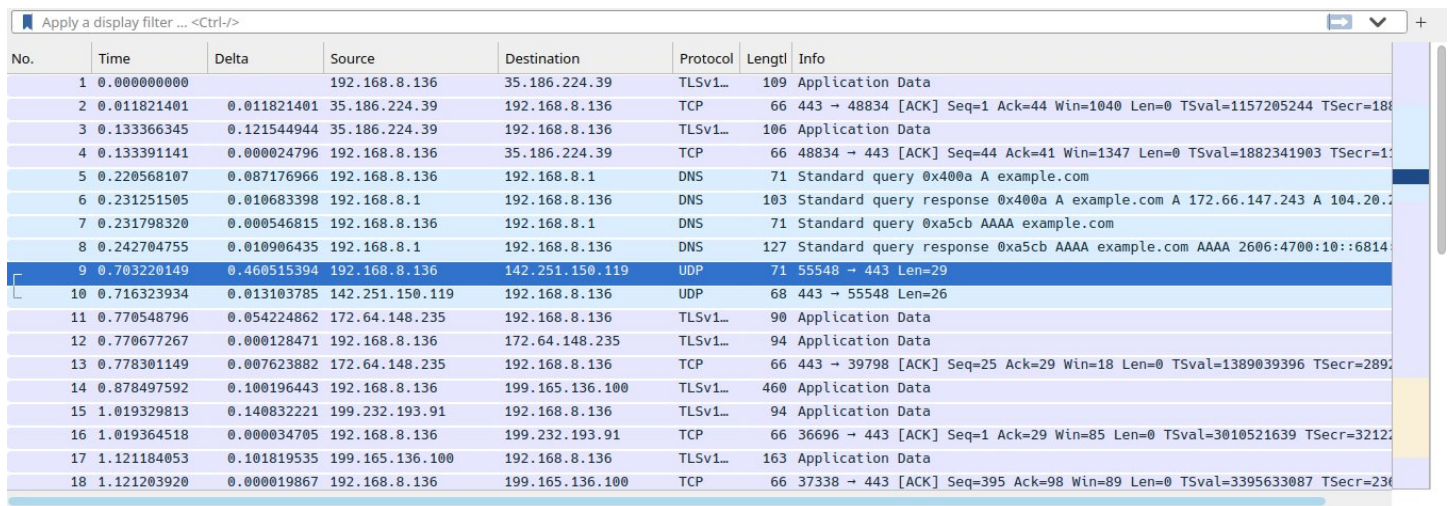
3. Now, can you relate the above scenarios to the well-known two transport layer protocols? Consider different scenarios including large file transfer and faster communication.

These two scenarios are identical to the two transport layer protocols TCP and UDP. UDP is more suitable when speed and low overhead matters more than reliability, as it does not establish a connection or guarantee the delivery or its ordering. This is useful for real time communication where small amounts of packet loss is considered to be acceptable. TCP on the other hand is more suitable for things like large file transfers, and other tasks where its important that the data gets delivered reliably and in the correct order. It uses sequencing, acknowledgments and retransmissions to make sure lost data is recovered.

Week 4 – Activity 2

1. Open Wireshark to capture packets, open the web browser and use an application that uses UDP (perhaps DNS). Stop the packet capture.

To started my packet capture on Wireshark.



No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000000		192.168.8.136	35.186.224.39	TLSv1...	109	Application Data
2	0.011821401	0.011821401	35.186.224.39	192.168.8.136	TCP	66	443 → 48834 [ACK] Seq=1 Ack=44 Win=1040 Len=0 TSval=1157205244 TSecr=186
3	0.133366345	0.121544944	35.186.224.39	192.168.8.136	TLSv1...	106	Application Data
4	0.133391141	0.000024796	192.168.8.136	35.186.224.39	TCP	66	48834 → 443 [ACK] Seq=44 Ack=41 Win=1347 Len=0 TSval=1882341903 TSecr=1
5	0.220568107	0.087176966	192.168.8.136	192.168.8.1	DNS	71	Standard query 0xa400a A example.com
6	0.231251505	0.010683398	192.168.8.1	192.168.8.136	DNS	103	Standard query response 0xa400a A example.com A 172.66.147.243 A 104.20.2
7	0.231798320	0.000546815	192.168.8.136	192.168.8.1	DNS	71	Standard query 0xa5cb AAAA example.com
8	0.242704755	0.010906435	192.168.8.1	192.168.8.136	DNS	127	Standard query response 0xa5cb AAAA example.com AAAA 2606:4700:10::6814
9	0.703220149	0.460515394	192.168.8.136	142.251.150.119	UDP	71	55548 → 443 Len=29
10	0.716323934	0.013103785	142.251.150.119	192.168.8.136	UDP	68	443 → 55548 Len=26
11	0.770548796	0.054224862	172.64.148.235	192.168.8.136	TLSv1...	90	Application Data
12	0.770677267	0.000128471	192.168.8.136	172.64.148.235	TLSv1...	94	Application Data
13	0.778301149	0.007623882	172.64.148.235	192.168.8.136	TCP	66	443 → 39798 [ACK] Seq=25 Ack=29 Win=18 Len=0 TSval=1389039396 TSecr=289
14	0.878497592	0.100196443	192.168.8.136	199.165.136.100	TLSv1...	460	Application Data
15	1.019329813	0.140832221	199.232.193.91	192.168.8.136	TLSv1...	94	Application Data
16	1.019364518	0.000034705	192.168.8.136	199.232.193.91	TCP	66	36696 → 443 [ACK] Seq=1 Ack=29 Win=85 Len=0 TSval=3010521639 TSecr=3212
17	1.121184053	0.101819535	199.165.136.100	192.168.8.136	TLSv1...	163	Application Data
18	1.121203920	0.000019867	192.168.8.136	199.165.136.100	TCP	66	37338 → 443 [ACK] Seq=395 Ack=98 Win=89 Len=0 TSval=3395633087 TSecr=236

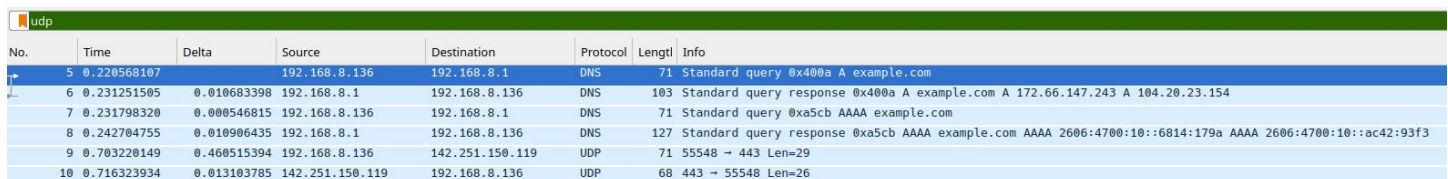
Simultaneously, to generate some UDP traffic, I ran the `nslookup` command (from the `net-binds` package).

```
[athena@pc ~]$ nslookup example.com
Server:      100.100.100.100
Address:     100.100.100.100#53

Non-authoritative answer:
Name:   example.com
Address: 172.66.147.243
Name:   example.com
Address: 104.20.23.154
Name:   example.com
Address: 2606:4700:10::6814:179a
Name:   example.com
Address: 2606:4700:10::ac42:93f3
```

2. Use the filter to display UDP packets (it is UDP segments).

Filter: `udp`



No.	Time	Delta	Source	Destination	Protocol	Length	Info
5	0.220568107		192.168.8.136	192.168.8.1	DNS	71	Standard query 0xa400a A example.com
6	0.231251505	0.010683398	192.168.8.1	192.168.8.136	DNS	103	Standard query response 0xa400a A example.com A 172.66.147.243 A 104.20.23.154
7	0.231798320	0.000546815	192.168.8.136	192.168.8.1	DNS	71	Standard query 0xa5cb AAAA example.com
8	0.242704755	0.010906435	192.168.8.1	192.168.8.136	DNS	127	Standard query response 0xa5cb AAAA example.com AAAA 2606:4700:10::6814:179a AAAA 2606:4700:10::ac42:93f3
9	0.703220149	0.460515394	192.168.8.136	142.251.150.119	UDP	71	55548 → 443 Len=29
10	0.716323934	0.013103785	142.251.150.119	192.168.8.136	UDP	68	443 → 55548 Len=26

3. Now you are ready to analyze UDP. The following questions can guide you to conduct your analysis. Make sure that you take enough screenshots and notes. You will use those to prepare your task submissions.

5	0.220568107	192.168.8.136	192.168.8.1	DNS	71	Standard query 0x400a A example.com
6	0.231251505	0.010683398	192.168.8.1	192.168.8.136	DNS	103 Standard query response 0x400a A example.com A 172.66.147.243 A 104.20.23.154
7	0.231798320	0.000546815	192.168.8.136	192.168.8.1	DNS	71 Standard query 0xa5cb AAAA example.com
8	0.242704755	0.010906435	192.168.8.1	192.168.8.136	DNS	127 Standard query response 0xa5cb AAAA example.com AAAA 2606:4700:10::6814:179a AAAA 2606:4700:10::ac42:93f3
9	0.703220149	0.460515394	192.168.8.136	142.251.150.119	UDP	71 55548 → 443 Len=29
10	0.716323934	0.013103785	142.251.150.119	192.168.8.136	UDP	68 443 → 55548 Len=26

```

> Frame 5: Packet, 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 192.168.8.1
> User Datagram Protocol, Src Port: 34735, Dst Port: 53
  - Source Port: 34735
  - Destination Port: 53
  - Length: 37
  - Checksum: 0x9210 [unverified]
  - [Checksum Status: Unverified]
  - [Stream index: 0]
  - [Stream Packet Number: 1]
  - [Timestamps]
  - UDP payload (29 bytes)
> Domain Name System (query)

```

5	0.220568107	192.168.8.136	192.168.8.1	DNS	71	Standard query 0x400a A example.com
6	0.231251505	0.010683398	192.168.8.1	192.168.8.136	DNS	103 Standard query response 0x400a A example.com A 172.66.147.243 A 104.20.23.154
7	0.231798320	0.000546815	192.168.8.136	192.168.8.1	DNS	71 Standard query 0xa5cb AAAA example.com
8	0.242704755	0.010906435	192.168.8.1	192.168.8.136	DNS	127 Standard query response 0xa5cb AAAA example.com AAAA 2606:4700:10::6814:179a AAAA 2606:4700:10::ac42:93f3
9	0.703220149	0.460515394	192.168.8.136	142.251.150.119	UDP	71 55548 → 443 Len=29
10	0.716323934	0.013103785	142.251.150.119	192.168.8.136	UDP	68 443 → 55548 Len=26

```

> Frame 6: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface enp9s0, id 0
> Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
> Internet Protocol Version 4, Src: 192.168.8.1, Dst: 192.168.8.136
> User Datagram Protocol, Src Port: 53, Dst Port: 34735
  - Source Port: 53
  - Destination Port: 34735
  - Length: 69
  - Checksum: 0x0d69 [unverified]
  - [Checksum Status: Unverified]
  - [Stream index: 0]
  - [Stream Packet Number: 2]
  - [Timestamps]
  - UDP payload (61 bytes)
> Domain Name System (response)

```

a. Pick one UDP packet from the filtered packets. Examine the number of fields that are in the header of the selected UDP packet. You can take a screenshot of the packet and name and explain the fields in the UDP packet.

I selected packet 5, which was a DNS query from 192.168.8.136 to 192.168.8.1. The UDP header here contains four fields:

- Source Port (34735), which identifies the sending application
- Destination Port (53), which identifies the receiving service,
- Length (37 bytes), which gives the total size of the UDP header and payload
- and the Checksum (0x9210), which is used for error detection

b. Can you identify the length of each UDP header field from this UDP packet? You can indicate the length in bytes. You may want to examine the displayed information in Wireshark's packet content field.

<pre> > Frame 6: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on > Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_ > Internet Protocol Version 4, Src: 192.168.8.1, Dst: 192.168.8.136 > User Datagram Protocol, Src Port: 53, Dst Port: 34735 - Source Port: 53 - Destination Port: 34735 - Length: 69 - Checksum: 0x0d69 [unverified] </pre>	<pre> 0000 b0 82 e2 0b 8d 70 94 83 c4 62 1d 53 08 00 45 00 0010 00 59 33 b5 40 00 40 11 75 05 c0 a8 08 01 c0 a8 0020 08 88 00 35 87 af 00 45 0d 69 40 0a 81 80 00 01 0030 00 02 00 00 00 00 07 65 78 61 6d 70 6c 65 03 63 0040 6f 6d 00 00 01 00 01 c0 0c 00 01 00 01 00 00 01 0050 1f 00 04 ac 42 93 f3 c0 0c 00 01 00 01 00 00 01 0060 1f 00 04 68 14 17 9a </pre>
---	--

Each UDP header field is 2 bytes in size (16 bits). Therefore, Source Port = 2 bytes, Destination Port = 2 bytes, Length = 2 bytes and finally, the Checksum = 2 bytes. Therefore, this makes the total UDP header size 8 full bytes.

This also makes sense because length – payload = header —> 69 - 61 = 8.

c. Can you explain the value in the Length field? What exactly this value indicates? You can verify your answer by examining captured UDP packets.

This field represents the total size of the UDP header + UDP payload. For example, packet 5 has a UDP length of 37 bytes, containing an 8 byte UDP header and 29 bytes of payload. Packet 6 also confirms this, with $69 = 8 + 61$ bytes as said above.

d. Is there a maximum number of bytes that we can include in UDP payload?

Yes, for IPv4, the maximum payload is 65,507 bytes^[3]. This is from: 65,535 bytes – 8 byte UDP header – 20 byte IP header.

e. Can you identify the protocol number given for UDP?

5	0.228568107		192.168.8.136	192.168.8.1	DNS	71	Standard query 0x400a A example.com
6	0.231251505	0.010683398	192.168.8.1	192.168.8.136	DNS	103	Standard query response 0x400a A example.com A 172.66.147.243 A 104.20.23.154
7	0.231798320	0.000546815	192.168.8.136	192.168.8.1	DNS	71	Standard query 0xa5cb AAAA example.com
8	0.242704755	0.010906435	192.168.8.1	192.168.8.136	DNS	127	Standard query response 0xa5cb AAAA example.com AAAA 2606:4700:10::6814:179a AAAA 2606:4700:10::ac42:93f3
9	0.703220149	0.460515394	192.168.8.136	142.251.150.119	UDP	71	55548 → 443 Len=29
10	0.716323934	0.013103785	142.251.150.119	192.168.8.136	UDP	60	443 → 55548 Len=26

Frame 6: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface enp9s0, id 0	0000
Ethernet II, Src: GLTechnologi_62:d1:d53 (94:83:c4:62:d1:d53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)	0010
Internet Protocol Version 4, Src: 192.168.8.1, Dst: 192.168.8.136	0020
0100 = Version: 4	0030
.... 0101 = Header Length: 20 bytes (5)	0040
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)	0050
Total Length: 89	0060
Identification: 0x33b5 (13237)	
> 010. = Flags: 0x2, Don't fragment	
...0 0000 0000 0000 = Fragment Offset: 0	
Time to Live: 64	
Protocol: UDP (17)	
Header Checksum: 0x7505 [validation disabled]	
[Header checksum status: Unverified]	
Source Address: 192.168.8.1	

It's 17 - Protocol: UDP (17). It can be seen inside the IPv4 header of these packets.

f. Examine two consecutive UDP packets your host sends/receives. Explain the relationship between the port numbers in these two packets.

In packet 5, my host sends a DNS query from source port 34735 to destination port 53. In packet 6, the DNS server replies from source port 53 back to destination port 34735. Therefore, the source and destination port numbers are reversed in the response, allowing the reply to be delivered back to the same UDP socket on my host. You can also see a very similar thing in packets 7 and 8 as well as 9 and 10.

g. Clear the cache and run another application such as MSTeams while using Web browsing. Capture packets of these two applications. Now you can analyze the captured UDP packets again. Do all UDP packets captured in Wireshark use the same port number? Explain your answer.

I joined a voice call in Discord while also browsing the web to generate some UDP traffic from multiple applications. In the capture, I could see large amounts of UDP traffic between my host on port 37054 and the remote host on port 19301, which was likely generated during the discord all. I could also see other UDP traffic like web/QUIC using port 443 (HTTP/3) and DNS traffic using port 53.

And to answer the question, no! The captured UDP traffic uses different ports depending on the application and the service. As mentioned above, DNS used port 53, web traffic used port 443 while discord voice traffic used different dynamically assigned ports. This allows the transport layer to multiplex and demultiplex UDP traffic to the correct application process accordingly.

Both the previous pcapng file as well as this file can be downloaded here. [Click Here](#).

Some screenshots from this capture are to be found in the next page.

398	12.448068678	0.025555615	192.168.8.136	162.159.128.233	QUIC	86	Protected Payload (KP0), DCID=0141d7346847370cb7427934304700dd1b1d6210
406	12.523796263	0.075727585	162.159.128.233	192.168.8.136	QUIC	867	Protected Pavload (KP0)

```

> Frame 398: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 162.159.128.233
User Datagram Protocol, Src Port: 48212, Dst Port: 443
  Source Port: 48212
  Destination Port: 443
  Length: 52
  Checksum: 0xecfe [unverified]
  [Checksum Status: Unverified]
  [Stream index: 2]
  [Stream Packet Number: 89]
  [Timestamps]
  UDP payload (44 bytes)
QUIC IETF

```

287	9.209285259	0.020498702	192.168.8.136	104.29.149.152	UDP	89	37054 → 19301 Len=47
288	9.251575567	0.042290308	192.168.8.136	104.29.149.152	UDP	89	37054 → 19301 Len=47

```

> Frame 287: Packet, 89 bytes on wire (712 bits), 89 bytes captured (712 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 104.29.149.152
User Datagram Protocol, Src Port: 37054, Dst Port: 19301
  Source Port: 37054
  Destination Port: 19301
  Length: 55
  Checksum: 0xc72e [unverified]
  [Checksum Status: Unverified]
  [Stream index: 5]
  [Stream Packet Number: 93]
  [Timestamps]
  UDP payload (47 bytes)
Data (47 bytes)

```

udp

No.

Time

Delta

Source

Destination

Protocol

Length

Info

93	1.925480774	0.000011161	192.168.8.136	104.29.149.152	UDP	116	42106 → 19301 Len=74
94	1.947526006	0.022045232	104.29.149.152	192.168.8.136	UDP	116	19301 → 37054 Len=74
96	1.950509551	0.002903545	104.29.149.152	192.168.8.136	UDP	116	19301 → 42106 Len=74
97	1.950539668	0.000030117	192.168.8.136	104.29.149.152	ICMP	144	Destination unreachable (Port unreachable)
101	1.976357865	0.025818197	192.168.8.136	162.159.128.233	QUIC	1292	Protected Payload (KP0), DCID=0141d7346847370cb7427934304700dd1b1d6210
102	1.976388132	0.000030267	192.168.8.136	162.159.128.233	QUIC	1286	Protected Payload (KP0), DCID=0141d7346847370cb7427934304700dd1b1d6210

```

> Frame 97: Packet, 144 bytes on wire (1152 bits), 144 bytes captured (1152 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 104.29.149.152
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
  Total Length: 130
  Identification: 0x3659 (13913)
  0000 .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x7c7c [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.8.136
  Destination Address: 104.29.149.152
  [Stream index: 14]
Internet Control Message Protocol
  Type: Destination unreachable (3)
  Code: 3 (Port unreachable)
  Checksum: 0xc446 [correct]
  [Checksum Status: Good]
  Unused: 00000000
  Internet Protocol Version 4, Src: 104.29.149.152, Dst: 192.168.8.136
User Datagram Protocol, Src Port: 19301, Dst Port: 42106
  Source Port: 19301
  Destination Port: 42106
  Length: 82
  Checksum: 0x6bf5 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 6]
  UDP payload (74 bytes)
Data (74 bytes)

```

0000	94 83 c4 62 1d 53 b0 82	e2 0b 8d 70 08 00 45 c0	...b S...p...E-
0010	00 82 36 59 00 00 40 01	7c 7c c0 a8 08 88 68 1d	...6Y...@...h-
0020	95 98 03 03 c4 46 00 00	00 00 45 00 00 66 00 00	...F...E...f...
0030	40 00 34 11 7f a1 68 1d	95 98 c0 a8 08 88 4b 65	@...h...Ke...
0040	a4 7a 00 52 6b f5 00 02	00 46 00 00 3a 37 32 31	z Rk...F...721
0050	30 2e 35 36 2e 32 34 33	2e 37 33 00 00 00 00 00	0.56.243 .73....
0060	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0080	00 00 00 00 00 00 00 00	00 00 00 00 00 00 46 dd	F-

The third image here is a bit of an interesting one. It shows an ICMP packet, and the reason it shows up there even when my display filter is set to `udp` is because it contains the original UDP packet that caused the error (can be seen when expanded, as seen in the screenshot). Wireshark sees that embedded UDP header, so the packet still matches the udp filter. In this case, the ICMP message is a Destination Unreachable (Port Unreachable) response, meaning the UDP packet was sent to a port where there was no application actively listening at that point. Since this was captured while I was in a Discord voice call, this could have happened because Discord had already closed or switched away from that temporary UDP port while another packet was still arriving – could be either, or some completely other yet identical reason.

Week 4 – Activity 3

Source Code (client):

```
1  from socket import *
2
3  serverName = "127.0.0.1"
4  serverPort = 11500
5
6  with socket(AF_INET, SOCK_DGRAM) as clientSocket:
7      message = input("Enter message: ")
8
9      clientSocket.sendto(message.encode(), (serverName, serverPort))
10
11     reply, serverAddress = clientSocket.recvfrom(2048)
12
13     print("Server reply:", reply.decode())
14
```

Source Code (server):

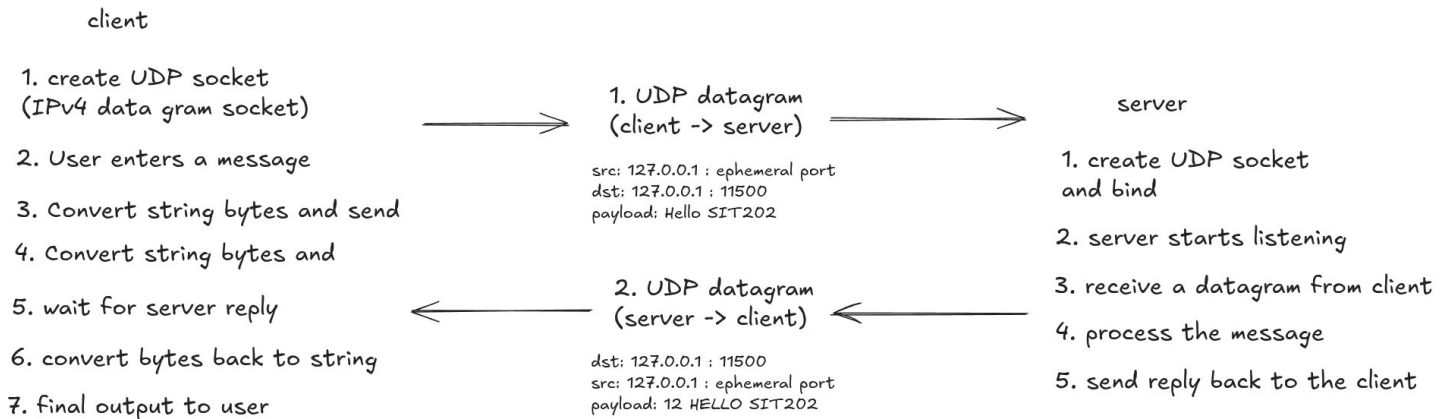
```
1  from socket import *
2
3  serverName = "127.0.0.1"
4  serverPort = 11500
5
6  with socket(AF_INET, SOCK_DGRAM) as serverSocket:
7      serverSocket.bind((serverName, serverPort))
8
9      print("The Server is Listening")
10
11     while True:
12         message, clientAddress = serverSocket.recvfrom(2048)
13
14         message = message.decode()
15         count = len(message)
16
17         print("Message received:", message)
18         print("Number of characters:", count)
19
20         reply = str(count) + " " + message.upper()
21
22         serverSocket.sendto(reply.encode(), clientAddress)
23
```

Execution:

```
● [athena@pc week4-activity3]$ python client.py
Enter message: Hello SIT202
Server reply: 12 HELLO SIT202
❖ [athena@pc week4-activity3]$

○ [athena@pc week4-activity3]$ python server.py
The Server is Listening
Message received: Hello SIT202
Number of characters: 12
□
```

A diagram of how this program works can be found below:



The client program creates a UDP docket using `SOCK_DGRAM`, then reads the message entered by the user as a string and converts it to bytes using the `.encode()` method to send it through the socket using `sendto()`. After that, it waits for the server to respond using `recvfrom()`.

The server program creates a UDP socket using `SOCK_DGRAM`, then binds the socket to port 11500 and waits for a message from the client using `recvfrom()`. After that, it decodes the received bytes back into a string, counts the number of characters, converts the message to uppercase and sends the response back to the client using `sendto()`.

The client uses an ephemeral port for communication. This is selected and assigned automatically by the operating system when the UDP socket is created.

Week 5 – Activity 1

1. Assume one of your group members sent you a message. However, you have never received it. What went wrong? How do you make sure that the message is properly received?

If the message was never received, the packet might have been lost somewhere during the transmission. I would use ACK and a timeout mechanism, where the receiver sends an acknowledgment after receiving the message, and if the sender does not get that ACK within that timeout window, it retransmits the message. This is basically the Stop and Wait ARQ mechanism^[4].

2. Assume you are trying to access a web page containing an image. Now, your job is to make sure that you get the image properly. The image cannot be received in one packet, and it will be broken down to 10 packets.

As the image is broken down into 10 packets, I would give each packet a sequence number from 1 to 10. This allows the client to identify the missing packets, detect duplicates and place everything back in the correct order before rebuilding the image. These are standardized concepts in reliable data transfer and is called sequencing and reassembly.

3. Partner up with one of your group members and develop a transport layer protocol to guarantee that the image is properly received by the client. One can act as the server where the webpage/image is stored and the other can act as the client who wants to receive the image (this means your group has 2 pairs of server-client networks).

I would develop a simple reliable ARQ (Automatic Repeat reQuest^[5]) based transport protocol between the server and the client. It would use sequence numbers, acknowledgements, error detection and retransmission to ensure that all the parts of this image are received correctly.

4. Write down protocol that you would use, including the communication between the client and server and all the messages that you are passing.

The server would send Packet 1, and once the client receives and verifies it, the client sends back ACK 1. The server then continues with the next packet. If an ACK is not received within the timeout window, the packet is sent again. This continues until all 10 packets have been acknowledged. This again is basically just Stop and Wait ARQ.

5. How do you guarantee that your protocol does the job as you expected?

I would use checksums to detect corrupted packets, sequence numbers to identify missing or duplicated packets, and ACKs with retransmission to recover from packet loss. The client will only rebuild the image after all 10 packets have been received correctly. These are basically some of the main ARQ components: error detection, receiver feedback and retransmission.

6. Once you have finalized your protocol, check how other group members developed their protocols.

I would compare our protocol with the other group members and mainly look at how their protocols handle packet loss, corrupted packets, acknowledgements, ordering and retransmissions. These would be the primary factors that determines how different/unique our answers are. But again, I'm doing this by myself.

7. With your group, discuss whether your protocol valid when you need to access a large file that might be broken down into 1000 packets. If not, what are the changes you would like to make in your protocol?

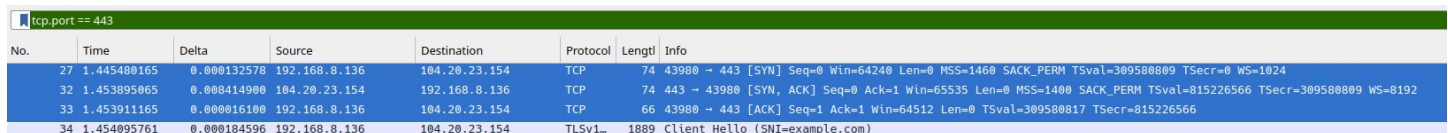
The same protocol could technically work for 100 packets. However, sending one packet and then waiting for one ACK every time (for each packet sent) would be very slow and inefficient. I would change it to a sliding window protocol like Selective Repeat. So, multiple packets can be sent before waiting for ACKs and only the actual missing packets need to be retransmitted. This is basically the Selective Repeat ARQ^[6].

Week 5 – Activity 2

Now you are familiar with Wireshark. Therefore, you should be able to analyze protocols and packets with less instructions provided. Today, you are going to analyze TCP.

1. You can analyze the TCP three-way handshake in Wireshark packet capture. Analyze the order of segments sent/received by two end systems to establish a TCP connection.
2. Analyze the segment headers used, their purpose and sizes.
3. Make sure you take the screenshots of TCP segments that will be used for your task submissions.

Below are some screenshots of what I believe is relevant here.



No.	Time	Delta	Source	Destination	Protocol	Length	Info
27	1.445480165	0.000132578	192.168.8.136	104.20.23.154	TCP	74	43980 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=309580809 TSecr=0 WS=1024
32	1.453895065	0.008414900	104.20.23.154	192.168.8.136	TCP	74	443 → 43980 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM TSval=815226566 TSecr=309580809 WS=8192
33	1.453911165	0.000016100	192.168.8.136	104.20.23.154	TCP	66	43980 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=309580817 TSecr=815226566
34	1.454095761	0.000184596	192.168.8.136	104.20.23.154	TLSv1	1889	Client Hello (SNI=example.com)

—> Next Page


```
> Frame 27: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 104.20.23.154
```

```
✓ Transmission Control Protocol, Src Port: 43980, Dst Port: 443, Seq: 0, Len: 0
```

```
  - Source Port: 43980
  - Destination Port: 443
  - [Stream index: 4]
  - [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete, DATA (15)]
  - [TCP Segment Len: 0]
  - Sequence Number: 0 (relative sequence number)
  - Sequence Number (raw): 1622656220
  - [Next Sequence Number: 1 (relative sequence number)]
  - Acknowledgment Number: 0
  - Acknowledgment number (raw): 0
  - 1010 .... = Header Length: 40 bytes (10)
  > .... 0000 0000 0010 = Flags: 0x002 (SYN)
  - Window: 64240
  - [Calculated window size: 64240]
  - Checksum: 0x490d [unverified]
  - [Checksum Status: Unverified]
  - Urgent Pointer: 0
  > Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  > [Timestamps]
  - [Client Contiguous Streams: 1]
  - [Server Contiguous Streams: 1]
```

```
> Frame 32: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface enp9s0, id 0
> Ethernet II, Src: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53), Dst: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70)
> Internet Protocol Version 4, Src: 104.20.23.154, Dst: 192.168.8.136
```

```
✓ Transmission Control Protocol, Src Port: 443, Dst Port: 43980, Seq: 0, Ack: 1, Len: 0
```

```
  - Source Port: 443
  - Destination Port: 43980
  - [Stream index: 4]
  - [Stream Packet Number: 2]
  > [Conversation completeness: Incomplete, DATA (15)]
  - [TCP Segment Len: 0]
  - Sequence Number: 0 (relative sequence number)
  - Sequence Number (raw): 2899414957
  - [Next Sequence Number: 1 (relative sequence number)]
  - Acknowledgment Number: 1 (relative ack number)
  - Acknowledgment number (raw): 1622656221
  - 1010 .... = Header Length: 40 bytes (10)
  > .... 0000 0001 0010 = Flags: 0x012 (SYN, ACK)
  - Window: 65535
  - [Calculated window size: 65535]
  - Checksum: 0x79d1 [unverified]
  - [Checksum Status: Unverified]
  - Urgent Pointer: 0
  > Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  > [Timestamps]
  > [SEQ/ACK analysis]
  - [Client Contiguous Streams: 1]
  - [Server Contiguous Streams: 1]
```

```

> Frame 33: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 104.20.23.154
✓ Transmission Control Protocol, Src Port: 43980, Dst Port: 443, Seq: 1, Ack: 1, Len: 0
  - Source Port: 43980
  - Destination Port: 443
  - [Stream index: 4]
  - [Stream Packet Number: 3]
> [Conversation completeness: Incomplete, DATA (15)]
  - [TCP Segment Len: 0]
  - Sequence Number: 1 (relative sequence number)
  - Sequence Number (raw): 1622656221
  - [Next Sequence Number: 1 (relative sequence number)]
  - Acknowledgment Number: 1 (relative ack number)
  - Acknowledgment number (raw): 2899414958
  - 1000 .... = Header Length: 32 bytes (8)
> .... 0000 0001 0000 = Flags: 0x010 (ACK)
  - Window: 63
  - [Calculated window size: 64512]
  - [Window size scaling factor: 1024]
  - Checksum: 0x4905 [unverified]
  - [Checksum Status: Unverified]
  - Urgent Pointer: 0
> Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
> [Timestamps]
> [SEQ/ACK analysis]
  - [Client Contiguous Streams: 1]
  - [Server Contiguous Streams: 1]

> Frame 34: Packet, 1889 bytes on wire (15112 bits), 1889 bytes captured (15112 bits) on interface enp9s0, id 0
> Ethernet II, Src: ASUSTekCOMPU_0b:8d:70 (b0:82:e2:0b:8d:70), Dst: GLTechnologi_62:1d:53 (94:83:c4:62:1d:53)
> Internet Protocol Version 4, Src: 192.168.8.136, Dst: 104.20.23.154
✓ Transmission Control Protocol, Src Port: 43980, Dst Port: 443, Seq: 1, Ack: 1, Len: 1823
  - Source Port: 43980
  - Destination Port: 443
  - [Stream index: 4]
  - [Stream Packet Number: 4]
> [Conversation completeness: Incomplete, DATA (15)]
  - [TCP Segment Len: 1823]
  - Sequence Number: 1 (relative sequence number)
  - Sequence Number (raw): 1622656221
  - [Next Sequence Number: 1824 (relative sequence number)]
  - Acknowledgment Number: 1 (relative ack number)
  - Acknowledgment number (raw): 2899414958
  - 1000 .... = Header Length: 32 bytes (8)
> .... 0000 0001 1000 = Flags: 0x018 (PSH, ACK)
  - Window: 63
  - [Calculated window size: 64512]
  - [Window size scaling factor: 1024]
  - Checksum: 0x5024 [unverified]
  - [Checksum Status: Unverified]
  - Urgent Pointer: 0
> Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
> [Timestamps]
> [SEQ/ACK analysis]
  - [Client Contiguous Streams: 1]
  - [Server Contiguous Streams: 1]
  - TCP payload (1823 bytes)
✓ Transport Layer Security
  - [Stream index: 2]
✓ TLSv1.3 Record Layer: Handshake Protocol: Client Hello
  - Content Type: Handshake (22)
  - Version: TLS 1.0 (0x0301)
  - Length: 1818
  ✓ Handshake Protocol: Client Hello
    - Handshake Type: Client Hello (1)
    - Length: 1814
  > Version: TLS 1.2 (0x0303)
    - Random: 4c5ecf3ab0055ce311ae03c96c679f991e175ddd296616119edfb49a9d67ac6b

```

The three main packets used in the TCP handshake were:

- Packet 27 (SYN): 192.168.8.136:43980 → 104.20.23.154:443
 - o TCP header: 40 bytes
 - o Raw Seq: 1622656220
 - o Flags: SYN
 - o MSS: 1460
 - o Window: 64240
- Packet 32 (SYN/ACK): 104.20.23.154:443 → 192.168.8.136:43980
 - o TCP header: 40 bytes
 - o Raw Seq: 2899414957
 - o Raw ACK: 1622656221
 - o Flags: SYN, ACK
 - o MSS: 1400
 - o Window: 65535
- Packet 33 (ACK): 192.168.8.136:43980 → 104.20.23.154:443
 - o TCP header: 32 bytes
 - o Raw Seq: 1622656221
 - o Raw ACK: 2899414958
 - o Flags: ACK

We also have Packet 34, which is the first actual data segment sent after the connection was established.

- Packet 34 (Data)
 - o TCP header: 32 bytes
 - o Payload: 1823 bytes
 - o Flags: PSH, ACK

The communication happened between my local host, the client (192.168.8.136) and the server (104.20.23.154). The client used the temporary TCP port 43980, which was automatically assigned by the operating system and the server used the port 443, which is the standard for HTTPS traffic.

Packet 34 is where actual application data starts being sent after the connection was established (after the three way handshake), with PSH, ACK set and 1823 bytes of TCP payload.

The main TCP segment headers noticeable here are^{[7][8]}:

- Source port: tells which application/socket is sending this segment.
- Destination port: tells which application/socket is receiving this segment.
- Sequence number: identifies the byte stream number of the first byte being sent in that segment. Note that this counts the bytes and not packets.
- Acknowledgment number: shows the sequence number of the next byte that the sender expects to receive from the other side.
- Header length: shows the size of the TCP header. SYN/SYN-ACK packets use 40 bytes (as they contain more TCP options) and ACK/data packets use 32 bytes.
- Flags: control how TCP handles the segment.
 - o SYN is used to establish a connection
 - o ACK is used to indicate that the acknowledgment number is valid
 - o PSH tells TCP to pass the received buffered data to the application without waiting for more data.

- Window Size: advertises how many more bytes the receiver is currently willing to accept. This is mainly used for flow control related features.
- Checksum: is used for error detection
- Urgent Pointer: is used with the URG flag to identify urgent data. In normal traffic, this is usually set to zero (0). When this is set (to 1), the it will be given a higher priority and will be processed by the receiving application immediately.
- TCP Options: extra parameters used by TCP, including MSS, SACK permitted, timestamps and window scaling. This is mainly why SYN packet headers are larger comparatively (40 vs 32).
- Payload Data: is not a part of the header. This is what carries the actual data. This can be seen in packet 34, which carries 1823 bytes of application data, containing the TLS ClientHello sent right after the TCP connection was established successfully.

Week 5 – Activity 3

Source Code (Client):

```
client.py > ...
1  from socket import *
2
3  serverName = "127.0.0.1"
4  serverPort = 11500
5
6  with socket(AF_INET, SOCK_STREAM) as clientSocket:
7      clientSocket.connect((serverName, serverPort))
8      message = input("Enter message: ")
9      clientSocket.send(message.encode())
10     reply = clientSocket.recv(2048)
11     print("Server reply:", reply.decode())
```

Source Code (Server):

```
server.py > ...
1  from socket import *
2
3  serverName = "127.0.0.1"
4  serverPort = 11500
5
6  with socket(AF_INET, SOCK_STREAM) as serverSocket:
7      serverSocket.bind((serverName, serverPort))
8      serverSocket.listen()
9
10     print("The Server is Listening")
11
12     # while True:
13     # -----
14     # put line 16-28 inside the while loop to allow multiple clients to connect
15     # and also multiple connections
16     # -----
17     connectionSocket, clientAddress = serverSocket.accept()
18     with connectionSocket:
19         message = connectionSocket.recv(2048)
20
21         message = message.decode()
22         count = len(message)
23
24         print("Message received:", message)
25         print("Number of characters:", count)
26
27         reply = str(count) + " " + message.upper()
28
29         connectionSocket.send(reply.encode())
```


Results (without While True):

```
[athena@pc week5-activity3]$ python client.py
Enter message: Hello SIT202
Server reply: 12 HELLO SIT202
[athena@pc week5-activity3]$
```

```
[athena@pc week5-activity3]$ python server.py
The Server is Listening
Message received: Hello SIT202
Number of characters: 12
[athena@pc week5-activity3]$
```

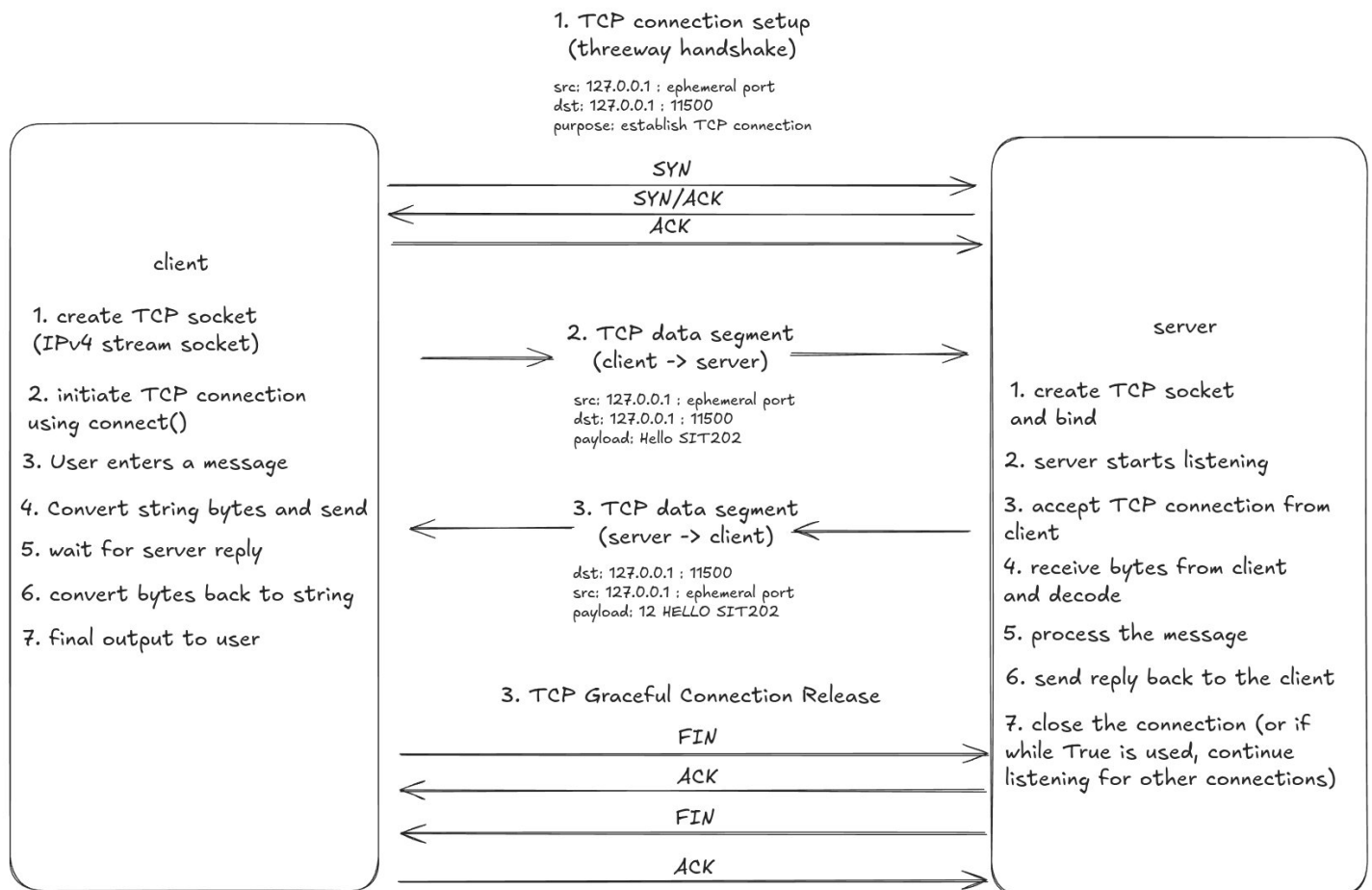
Results (with While True):

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[athena@pc week5-activity3]$ python client.py
Enter message: Hello SIT202
Server reply: 12 HELLO SIT202
[athena@pc week5-activity3]$
```

```
[athena@pc week5-activity3]$ python server.py
The Server is Listening
Message received: Hello SIT202
Number of characters: 12

```

Diagram:



The client program here creates a TCP socket using `SOCK_STREAM` and connects to the server using `connect()` before reading the message entered by the user as a string and converting it to bytes using the `.encode()` method to send it through the socket using `sendall()`. After that, it waits for the server to respond using `recv()`.

The server program here creates a TCP socket using `SOCK_STREAM` and binds the socket to port 11500, starts listening using `listen()` and accepts the client connection(s) using `accept()`. After that, it receives the message using `recv()`, then decodes the received bytes into a string, counts the number of characters it has, converts the message to uppercase and sends the response back to the client using `sendall()`.

The client here uses an ephemeral port for communication and is selected and assigned automatically by the operating system. Using a `while true` loop on the server allows it to go back to `accept()` after one client connection has finished and has been dealt with, meaning the server can keep accepting and processing new TCP connections instead of stopping after the first client. Note that these connections are still handled one at a time.

A `with` statement has been used (as context managers) for sockets, meaning python will automatically close the socket properly once that block has finished running. Once the client and the server are done communicating with each other, TCP can then gracefully release the connection using the normal FIN, ACK, FIN, ACK connection termination process instead of leaving the connection open. One side first sends a FIN to say that it has finished sending data, the other side responds with an ACK, and once it is also finished it sends its own FIN, which is acknowledged with the final ACK. This allows both the sides to close their direction of the connection separately and make sure the connection is properly terminated after the data transfer is finished^[9].

References

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021, ch. 3.
- [2] E. Modiano, "ARQ protocols: Go Back N and SRP," *16.36 Communication Systems Engineering*, Massachusetts Institute of Technology OpenCourseWare, Spring 2009. [Online]. Available: https://ocw.mit.edu/courses/16-36-communication-systems-engineering-spring-2009/9b06e175ee2b383dc35490581a45267a_MIT16_36s09_lec18.pdf. [Accessed: Aug. 15, 2026].
- [3] "User Datagram Protocol," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/User_Datagram_Protocol. [Accessed: Aug. 15, 2026].
- [4] GeeksforGeeks, "Stop and Wait ARQ," Feb. 26, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/stop-and-wait-arq/>. [Accessed: Aug. 15, 2026].
- [5] GeeksforGeeks, "Automatic Repeat Request (ARQ)," Dec. 30, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/what-is-arq-automatic-repeat-request/>. [Accessed: Aug. 15, 2026].
- [6] GeeksforGeeks, "Selective Repeat – Sliding Window Protocol," Dec. 30, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/sliding-window-protocol-set-3-selective-repeat/>. [Accessed: Aug. 15, 2026].
- [7] W. Eddy, "Transmission Control Protocol (TCP)," RFC 9293, Aug. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9293/>. [Accessed: Aug. 15, 2026].
- [8] Y. Han, K. Wang, Y. Zhao, and K. Yang, "Smart-TCP: An agentic AI-based autonomous and adaptive TCP protocol," *arXiv preprint arXiv:2512.00491*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.00491>. [Accessed: Aug. 15, 2026].
- [9] GeeksforGeeks, "TCP Connection Termination," Nov. 6, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/tcp-connection-termination/>. [Accessed: Aug. 15, 2026].